

CACHE MEMORY DESIGN

Contents

Introduction 2

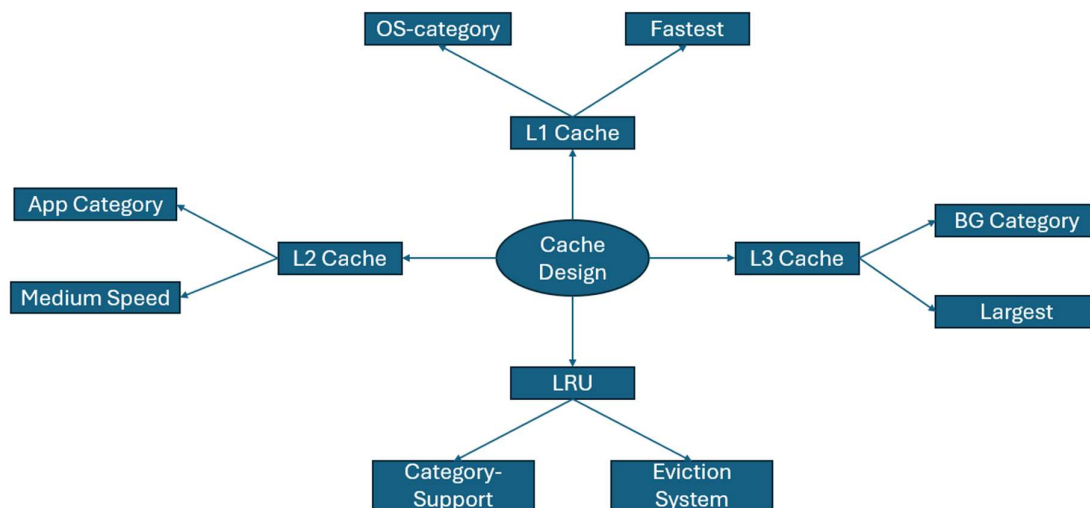
Cache Category-Based Memory 3

Cache Design Trade-offs 4

Introduction

This Cache Memory Architecture is designed to help improve the performance of computers. It is also made as an extension of the RAM Memory Architecture design. This cache design was created because the CPU fetches the most frequently used instructions. The most frequently used instructions are stored in cache as it is directly in the CPU and has limited storage space. This cache design mainly uses pre-existing systems like LRU and the current levels. This design uses the category-based memory only since there is no need for encrypting cache. This documentation will explain the features and trade-offs of this cache design.

This cache design, like the RAM memory architecture cannot promise perfect performance. Instead, this cache design can promise to further improve performance alongside the RAM memory architecture. This cache design may take years to implement along with the RAM memory architecture. Engineers and OS vendors may need to work together to implement this cache design. This cache design's categories can be adapted to fit specific computing systems.



The spider diagram above shows a layout of the Cache Memory Architecture. For those that aren't familiar with computing architecture, LRU stands for least recently used. The same 3 categories as the RAM memory architecture are used here since cache and RAM perform the same task. This task is storing instructions which the CPU fetches, decodes, and then executes.

Cache Category-Based Memory

- Cache category-based memory is where instructions within cache are sorted by categories instead of sharing levels.
- This cache category-based memory uses the two pre-existing systems of LRU and L1, L2 and L3 cache.
- This Cache design can be adapted for different use cases just like general purpose, gaming or AI workloads.
- For L1 Cache, it should store the most frequent OS category instructions as it is the fastest but also smallest cache
- This is important because OS instructions need instant access with no waiting which L1 cache can provide.
- For L2 Cache, it should store the most frequent App category instructions.
- This is because L2 cache has the second fastest speed but also second largest size compared to the other two levels.
- This is important because application instructions need quick but not instant access compared to the operating system in general purpose systems.
- For L3 Cache, it should store the most frequent background category instructions.
- This is because L3 cache has the biggest amount of space but also the slowest access times.
- This is important because background processes do not require as much speed compared to applications and OS processes.
- For the case, where data is not in cache, the CPU can skip straight to RAM instead of searching other cache levels.
- This is possible because each cache stores a specific type of data and if the data is not found in the dedicated then it won't be in the other levels.
- For the write-back, the CPU must ensure to update cache by category to ensure that cache data doesn't become stale.
- This category-based memory sorting for cache benefits performance as it removes the competition for resources within cache.
- This cache design uses LRU for its eviction policy.
- LRU must be configured for this design to be applied within each category and not globally.
- LRU would evict the least frequently used OS data in L1 cache, App data in L2 cache and BG data in L3 cache.
- This is important to ensure that the cache can stay updated with the most frequently used instructions.
- The categories in each level can be pre-configured based on the system for example in gaming systems L1 cache would be used for gaming.
- This is important because gaming data needs more instant access within gaming systems compared to the operating system.

Cache Design Trade-offs

- This cache design has trade-offs just like any other system I have built.
- There are 3 main trade-offs I've identified.
- The first trade-off is Heat.
- This trade-off exists because the CPU can access specific instructions from each level of cache leading to faster fetching meaning more instructions executed quickly.
- This means that the CPU will heat up as heat is produced as more work is done by the CPU.
- This trade-off can be managed through cooling systems and limiting clock-speed for the CPU.
- The second trade-off is Application/Background Performance.
- This trade-off exists for two reasons.
- The first reason is that application instructions are stored in L2 cache which is medium speed and size.
- This means that some applications may run slower compared to operating system processes.
- The second reason is that background process instructions are stored in L3 cache which is the largest but slowest.
- This means that some background processes like updates and security software may have slower performance.
- This trade-off can be managed by adapting categories per level based on use case.
- For example, Applications may be in L1 cache for gaming systems.
- The third trade-off is cache misses.
- This trade-off exists because sometimes the instructions that the CPU is looking for are not in cache.
- This means the CPU searches all levels of cache before going to RAM which can be slow and time-consuming.
- This trade-off can be managed through a system where if the instruction is not stored in a specific level, then the CPU immediately heads to RAM instead of searching the other levels of cache.
- This improves performance because it saves time and allows the CPU to immediately fetch the instruction from RAM.
- This is also supported by category-based memory in RAM as the CPU can still fetch instructions quickly.